

programmation, car un langage est souvent conçu pour répondre à des besoins ou des problèmes spécifiques : on programme en assembleur pour écrire un driver de carte graphique afin d'être au plus proche du fonctionnement de la machine, le langage SQL est plus adapté pour réaliser des opérations avec une base de données ...

Définition

Paradigme : manière de programmer suivant un ensemble de règles. (*paradigme = modèle*)

On peut donc regrouper les langages de programmation dans plusieurs familles qui correspondent à des schémas de pensée différents : ce sont les paradigmes de programmation dont voici les principaux :

- Impératif
- Orienté Objets
- Fonctionnel
- Logique
- Événementiel
- Orienté requêtes
- Orienté contraintes
- Synchrone
- Spécification
- Flot de données
- Modélisation
- Programmation parallèle

Certains langages appartiennent à plusieurs de ces familles, c'est par exemple le cas de Python (ainsi que de C++, Ruby, Ocaml...) : ce sont des langages **multi-paradigmes**, dits aussi généralistes qui peuvent s'adapter à beaucoup de situations et sont conçus pour permettre de programmer le plus d'applications possibles.

Comme les langages de programmation utilisés aujourd'hui ne sont pas ceux d'hier ni peut-être ceux de demain, il est plus important de maîtriser les concepts des paradigmes de programmation, plutôt que de chercher à apprendre un très grand nombre de langages.

Dans la suite de cette activité, nous allons présenter des paradigmes déjà utilisés en NSI : impératif, orienté objets et événementiel ; puis découvrir de nouveaux paradigmes : programmation parallèle, logique et fonctionnel.

Programmation impérative

C'est le paradigme de programmation le plus classique, le premier historiquement, celui avec lequel vous avez probablement découvert la programmation. On y trouve toutes les structures comme les boucles (*for*, *while*), les conditionnelles (*if...*), les variables, les tableaux... Les opérations sont une suite d'instructions exécutées par l'ordinateur pour modifier l'état du programme, défini par le contenu de la mémoire à un instant donné, comme une recette de cuisine. **La brique élémentaire est l'instruction.**

La grande majorité des langages de programmation est impérative, car les micro-processeurs ont été conçus pour exécuter des séquences d'instructions.

Quelques langages impératifs : langage machine, Fortran, Basic, C, Python, PHP, Java.

Programmation orientée objet

Vous avez découvert cette année la programmation orientée objet (**POO**) qui consiste en la création et l'interaction de **briques logicielles appelées objets**. Un objet peut représenter à peu près n'importe quoi : un concept, une idée, une personne, un pays, une carte, un jeu... Il est doté d'**attributs** (variables, propriétés) et de **méthodes** pour agir sur lui-même ou interagir avec d'autres objets. On modélise ainsi des éléments réels par des objets virtuels.

Quelques langages orientés objets : Smalltalk, C++ qui est un C amélioré sorti en 1983 avec ajout des classes et des objets, Python, PHP, Java, Kotlin, OMT, UML.

On rappelle qu'une notion clé des langages orientés objets est **l'encapsulation**. Ce concept consiste à interdire, depuis l'extérieur de la classe, tout accès direct, que ce soit en lecture ou en écriture, aux variables d'instances de la classe. La seule façon d'y accéder consiste à passer par des méthodes.

Il existe notamment 2 intérêts fondamentaux à l'encapsulation :

- Garantir l'état des instances de la classe. Par exemple, interdire des points avec une ordonnée négative ;
- Permettre au créateur d'une classe de modifier la structure interne de la classe sans que cela n'impacte les utilisateurs de la classe.

En respectant l'encapsulation, si l'on souhaite donner accès à un attribut depuis l'extérieur de la classe, on utilisera des méthodes dédiées, que l'on appelle des **accesseurs** :

- Un **accesseur en lecture** est une méthode qui, se contente de renvoyer la valeur d'une variable d'instance. Par convention, son nom est le nom de la variable d'instance précédé de *get* ;
- Un **accesseur en écriture** (ou mutateur) est une méthode qui se contente de modifier la valeur d'une variable d'instance, avec éventuellement une vérification de la nouvelle valeur. Par convention, son nom est le nom de la variable d'instance précédé de *set*.

➔ Exercice 1 :

Voilà l'implantation de la **classe Point en Python** :

```
from math import sqrt

class Point:
    def __init__(self, initX, initY):
        self.x = initX
        self.y = initY

    def get_x(self):
        return self.x

    def get_y(self):
        return self.y
```

```

def set_y(self, nouvelleValeur):
    if (nouvelleValeur >= 0):
        self.y = nouvelleValeur

def translater(self, dx, dy):
    self.x += dx
    self.y += dy

def module(self):
    return sqrt(self.x*self.x + self.y * self.y)

def __str__(self):
    return "(" + str(self.x) + "," + str(self.y) + ")"

p1 = Point(3,4)
print(p1)
d = p1.module()
print(d)
p1.translater(1,1)
print(p1)
print(p1.get_x())
p1.y = 2
print(p1)
p1.y = -2
print(p1)

```

1. Teste ce programme sous PyZO. Que permet le langage Python ? Est-ce gênant ? Explique.

2. Corrige le programme pour qu'il respecte le concept fondamental d'encapsulation.

À retenir

En **Python**, l'encapsulation est purement conventionnelle : on peut toujours accéder aux variables d'instance d'un objet à l'extérieur de la classe. Mais c'est **fortement déconseillé**. Le langage Python multi-paradigmes permet de presque tout faire mais est très permissif

Pour d'autres langages, l'encapsulation peut être forcée ou utilisable via des déclarations explicites, ce qui rend le code plus solide :

- en **Kotlin**, l'encapsulation est forcée : les variables d'instance ne sont pas accessibles directement à l'extérieur de la classe ;
- en **Java**, l'encapsulation est utilisable via des déclarations explicites : il est en effet possible d'associer des *droits* aux attributs et aux méthodes. Pour mettre en œuvre l'encapsulation, on associera aux attributs le droit *privé*, et on associera aux méthodes qui doivent être accessibles à l'extérieur de la classe le droit *public*.

Programmation événementielle

Vous avez étudié l'an passé le langage **Javascript**, spécifiquement conçu pour rendre les pages web plus interactives. Il est donc optimisé pour répondre aux événements qui peuvent survenir sur une page web. Javascript obéit – entre autres – au paradigme de programmation événementielle.

Le langage **Scratch** est au autre exemple de langage événementiel dans lequel on associe des blocs de code à des actions de l'utilisateur.

Le développement d'interfaces graphiques (par exemple TKinter , Pygame en Python) fait largement appel à la programmation événementielle. Cela consiste à associer du code pour qu'il soit exécuté lorsque survient un événement.

➔ Exercice 2 :

Compléter le code ci-dessous (sous pyzo) pour qu'à chaque clic de bouton, le nombre affiché sur celui-ci soit incrémenté.

```
import tkinter as tk

def gerer_click(ev) :
    # code à compléter

fenetre=tk.Tk()
b=tk.Button(fenetre, text='0')
b.pack()
# code à compléter
fenetre.title("Cliquer sur le bouton")
fenetre.geometry("800x400")
fenetre.mainloop()
```

Aide : - chaque *widget* possède une opération `bind()` qui permet d'associer une fonction à un événement (de la souris ou du clavier) déclenché pour ce *widget*.

- L'événement `<Button-1>` permet de détecter un clic sur le bouton gauche de la souris.

<https://zestedesavoir.com/tutoriels/1729/programmation-avec-tkinter/programmation-evenementielle-avec-tkinter/>

Programmation parallèle

Pour les programmes nécessitant beaucoup de calculs (simulation par exemple), il est rapidement devenu nécessaire de lancer plusieurs tâches en même temps. Pour cela, il a fallu développer des machines faites de nombreux processeurs. Au départ, cela ne concernait que les super-calculateurs, mais avec le développement des **processeurs multi-core**, il est devenu possible d'exécuter plusieurs tâches en même temps sur un ordinateur personnel.

Cependant, l'intérêt d'effectuer plusieurs tâches en même temps n'est pas lié qu'à cela. Par exemple, l'humain étant particulièrement long à taper sur un clavier, un logiciel de traitement de texte passe 99 % de temps à ne rien faire. Il est donc intéressant, pendant ses longues périodes d'inactivité, de laisser d'autres logiciels s'exécuter. Mais si l'un de ces logiciels est un programme de calcul intensif, le système ne va pas attendre la fin de son exécution avant de repasser la main au traitement de texte. Le système va ainsi partager le temps du micro-processeurs entre les différents logiciels, ou plus exactement entre les différents *processus*.

Utiliser plusieurs processus peut également servir au sein d'une même application. Par exemple, si votre application graphique doit lancer un calcul qui prend plusieurs secondes, il n'est pas concevable, que cela bloque l'application. Aussi, on peut être amené, au sein d'une même application, à créer plusieurs *threads*, qui sont des sortes de processus légers.

En ce qui concerne l'exécution de tâches en parallèle, certains langages, comme **ADA**, comportent des instructions spéciales pour décrire ce genre de traitement, mais ces langages sont peu nombreux. Dans la plupart des langages, c'est notamment le cas de Python, il existe des bibliothèques qui permettent de créer et lancer des threads.

➔ Exercice 3 :

Voici un programme Python constitué de 3 threads différents programmés pour fonctionner simultanément : le thread principal, et 2 threads, créés grâce au constructeur `Thread()`, qui prend en paramètre la fonction qui devra être exécutée par le thread en question, et lancés par l'appel à la méthode `start()`. Ces deux derniers threads partagent une donnée, la variable globale `c` : dans un cadre plus général, les threads doivent partager des données pour pouvoir échanger de l'information.

```
from threading import Thread
from time import sleep
from random import randint

c = 0
def traitementA():
    global c
    i = 0
    while i < 10000000:
        c = 1
        #sleep(randint(1,10)/10)
        if c != 1:
            print("***** PROBLEME POUR A *****")
        i += 1

def traitementB():
    global c
    i = 0
    while i < 10000000:
        c = 2
        #sleep(randint(1,10)/10)
        if c != 2:
            print("***** PROBLEME POUR B *****")
        i += 1

t1 = Thread(target=traitementA)
t2 = Thread(target=traitementB)

t1.start()
```

```

t2.start()

for i in range(10):
    print("Thread principal")
    sleep(0.01)
print("Thread principal terminé")

```

1. Teste ce programme sous PyZO.

Obtient-on un affichage conforme au code des fonctions `traitementA()` et `traitementB()` ?
 Explique ce qui se passe d'après toi.

2. On va résoudre le problème à l'aide d'une opération atomique appelée *Test and set* : il s'agit d'une opération qui ne peut pas être interrompue (d'où le qualificatif *atomique*) qui teste la valeur d'une variable puis la modifie en fonction du résultat du test. Cette opération existe comme opération de base des microprocesseurs, et son utilisation se traduit par l'utilisation de **verrous** en programmation.

Un **verrou** est un objet possédant 2 états : *libre* et *verrouillé*. Un thread qui verrouille un verrou libre continue son exécution normalement. Mais un thread qui tente de verrouiller un verrou déjà verrouillé est bloqué jusqu'à ce que le verrou ait été déverrouillé.

Lorsqu'un verrou est déverrouillé, l'un des threads en attente de se verrouiller est débloqué et peut maintenant verrouiller ce verrou.

Ainsi, en utilisant des verrous, on peut déterminer des **zones d'exclusion mutuelle** ou **sections critiques** qui ne peuvent être exécutées que par 1 thread à la fois. Le problème présenté précédemment peut donc être résolu en utilisant des verrous.

En Python, la classe `Lock` du module `threading` permet de créer des verrous. Ceux-ci peuvent être verrouillés et déverrouillés respectivement par les méthodes `acquire()` et `release()`. (un peu de POO en programmation parallèle !)

Corrige le programme précédent ainsi :

```

from threading import Thread, Lock
from time import sleep
from random import randint

c = 0
def traitementA():
    global c
    i = 0
    while i < 10000000:
        verrou.acquire()
        c = 1

```

```

        if c != 1:
            print("***** PROBLEME POUR A *****")
            verrou.release()
            i += 1

def traitementB():
    global c
    i = 0
    while i < 1000000:
        verrou.acquire()
        c = 2
        if c != 2:
            print("***** PROBLEME POUR B *****")
            verrou.release()
            i += 1

verrou = Lock()
t1 = Thread(target=traitementA)
t2 = Thread(target=traitementB)
t1.start()
t2.start()

for i in range(10):
    print("Thread principal")
    sleep(0.01)
print("Thread principal terminé")

```

Que constates-tu ?

Programmation logique

La programmation logique est un paradigme de programmation déclaratif très différent de tout ce que vous avez pu voir jusqu'à présent. Il repose essentiellement sur la logique des **prédicats** (ou logique du premier ordre). Il y a peu de langages mettant en œuvre la programmation logique. Ils sont spécialement conçus pour traiter des problèmes logiques, voire faire des preuves mathématiques. On peut en citer trois : **Prolog**, **mercury** et **Coq**. Prolog restant malgré tout la référence, c'est ce langage qu'on va utiliser ici.

Prolog est un langage créé en 1972 par un Français : Alain Colmerauer. C'est un langage principalement utilisé dans le domaine de l'intelligence artificielle, mais aussi dans le traitement du langage.

Pour utiliser Prolog, tu peux bien sûr installer [swi-prolog](#) sur ta machine, mais le plus simple (et **conseillé dans un 1^{er} temps**) pour tester les exemples de ce cours est de te rendre sur l'interpréteur en ligne [swish](#).

Si tu as la chance de travailler sous **Linux** (par exemple Ubuntu), tout est plus rapide !! Il suffit de saisir dans le terminal **sudo apt-get install swi-prolog**. Le code est saisi dans un fichier texte (avec **gedit** par exemple) et enregistré en **.pl**. Dans le terminal, après avoir lancé **swipl** pour appeler l'interpréteur SWI-Prolog, aller dans le répertoire contenant le fichier, saisir **[nom_du_fichier]**. puis poser les questions sur les objets et leurs relations.

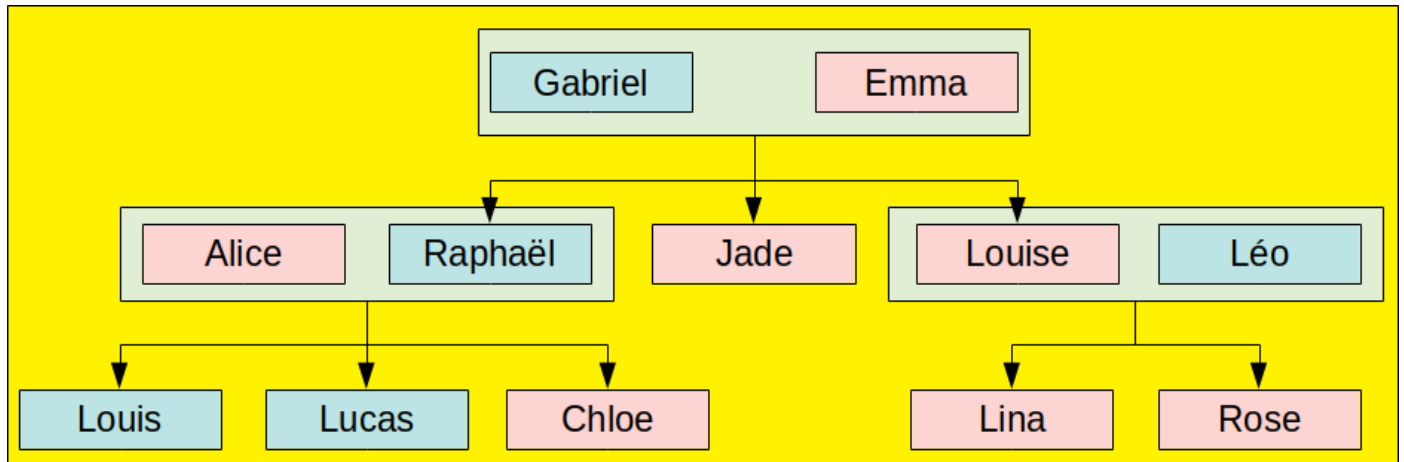
Dans Prolog, un programme est en fait une **base de faits et règles**, et le programme est en quelque sorte exécuté en **posant une question** dans l'interpréteur.

Programmer signifie donc :

- Spécifier des faits (vérifiés) sur les objets et leurs relations ;
- Définir des règles sur les objets et leurs relations ;
- Poser des questions sur les objets et leurs relations.

Exemple : un arbre généalogique

Pour illustrer le fonctionnement de Prolog, un exemple très simple sur la manipulation de l'arbre généalogique suivant (très, trop ... conventionnel !) :



Dans notre base de faits, nous allons énoncer différents types d'information pour représenter cet arbre :

- Le sexe (masculin ou féminin) de chacun ;
- Les couples (qui est marié avec qui) ;
- Les liens de filiation (qui est enfant de qui).

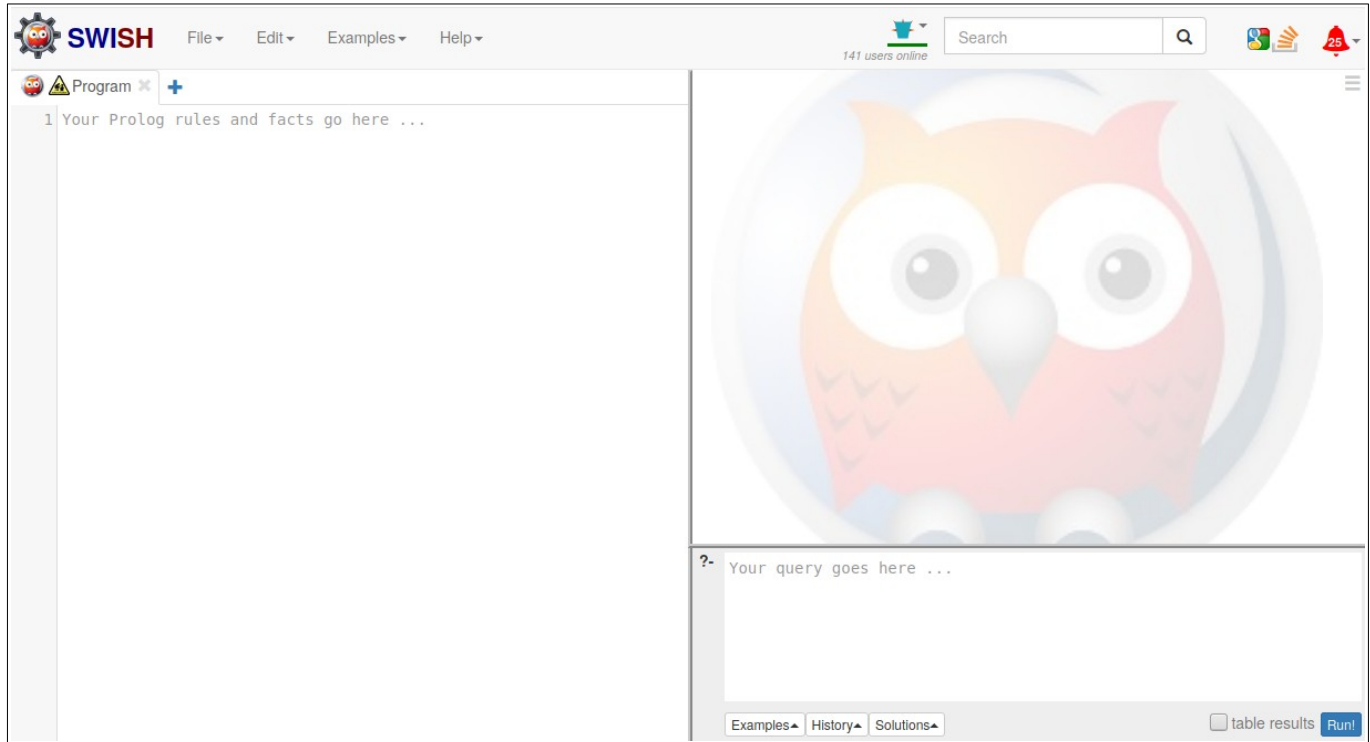
Par exemple, pour énoncer le fait que Gabriel est de sexe masculin, on va tout simplement écrire :

`masculin(gabriel).`

Plusieurs remarques :

- On n'a jamais déclaré ce qu'était "masculin" ou "gabriel" ;
- On n'a surtout pas mis de majuscule à "gabriel" (ni à "masculin" d'ailleurs) ;
- La ligne se termine par un point (".") ;
- "masculin" est un prédicat = « ce qui est affirmé comme vrai pour la valeur "gabriel" dans l'énoncé "masculin(gabriel)" »
- En Prolog règne l'**hypothèse du tiers exclu** : cela signifie que tout ce qui n'est pas explicitement vrai est faux. Du coup, si ma base de faits et règles se limite à cette seule déclaration, cela signifie en même temps que je déclare qu'Emma n'est pas de sexe masculin, mais aussi que Raphaël n'est pas de sexe masculin.

Lorsque tu vas sur le site de [swish](https://swish2.fr/) et que tu cliques sur “Programme”, s’ouvre l’interface suivante :



Dans cette interface :

1. La partie gauche est dédiée à la saisie de la base de faits et règles ;
2. La partie en bas à droite est la partie dédiée à la saisie des questions ;
3. La partie en haut à droite est destinée à l’affichage des résultats ;
4. Le bouton **Run!** en bas à droite sert à poser la question saisie.

Remplis maintenant la base de fait avec `masculin(gabriel)`. Pour interroger la base, nous allons maintenant poser des questions dans la zone dédiée. Par exemple, pour savoir si `gabriel` est de sexe masculin, il est possible de saisir la question suivante : `masculin(gabriel) ..`

En cliquant sur le bouton **Run!**, on obtient la réponse **True**. Par ailleurs, il est important de distinguer la différence sémantique entre le `masculin(gabriel)` . saisi à gauche, qui correspond à un **fait** (on sait que gabriel est de sexe masculin) et le `masculin(gabriel) ..`, qui correspond à une **question** (est-ce-que je peux déduire de ma base que `gabriel` est de sexe masculin ?).

Question 1 : Vérifie que `emma` n’est pas de sexe masculin et que `raphael` n’est pas de sexe masculin non plus.

Notion de variable logique

En Prolog, une variable logique est représentée par un mot commençant par une majuscule. Elle peut être utilisée dans une question, ce qui revient à demander de résoudre une équation en trouvant une valeur pour la variable qui permette de satisfaire la question.

Ajoute dans la base de fait :

```
masculin(raphael).
masculin(leo).
masculin(louis).
masculin(lucas).
```

Puis saisis la question suivante : masculin(X) .

Question 2 : Vérifie que Prolog propose toutes les valeurs possibles jusqu'à ce qu'il n'y ait plus de solution via le bouton **Next**.

Une variable logique peut également être utilisée dans un fait ou dans une règle. Il faut alors la considérer comme une variable quantifiée **universellement**.

Ainsi, si l'on rajoute le fait suivant à notre base : **personne(P)** , cela revient à écrire : pour toute valeur P, **personne(P)** est vraie.

Prédicat avec un ou plusieurs paramètres

Pour représenter que Raphaël est l'enfant d'Emma, on va rajouter un nouveau prédicat, **enfant**, qui sera un prédicat à 2 paramètres (on parle de prédicat d'**arité 2**) : l'un correspondant à l'enfant (Raphaël), l'autre correspondant au parent (Emma). On représente en général l'arité du prédicat en faisant suivre le nom du prédicat par un "/" suivi de l'arité. Ainsi, donc notre cas, on parlera des prédicats **masculin/1** et **enfant/2**.

Par convention, on met souvent en premier argument celui qui correspond à la "réponse" que suggère le nom du prédicat. On peut donc étoffer notre base de faits en rajoutant le fait suivant :

`enfant(raphael, emma).`

Question 3 : Complète la base de faits et règles avec les prédicats **masculin/1**, **feminin/1**, **enfant/2** et **mari/2** pour terminer la représentation de l'arbre généalogique.

Les règles

Jusqu'à présent, notre base de faits et règles ne contenait que des faits, ce qui limitait beaucoup son utilisation. Nous allons maintenant lui rajouter des *règles*.

Nous aimerions par exemple pouvoir savoir qui est la mère de Raphaël. C'est clairement une information qui n'est pas présente telle qu'elle dans notre base. Mais c'est une information que nous (humains) pouvons déduire. En effet, nous savons qu'une mère, c'est un parent de sexe féminin. Ou encore, pour qu'une personne P soit mère d'une personne E, il faut que E soit l'enfant de P **et** que P soit de sexe féminin. C'est même une condition suffisante pour tout couple (P, E). En logique, on noterait cela ainsi :

Pour tout couple (P,E). (feminin(P) vrai et enfant(E,P) vrai) implique mere(P, E)

Ceci constitue une *règle* que nous allons pouvoir rajouter à notre base de faits et règles sous la forme suivante :

`mere(P,E):- feminin(P), enfant(E, P).`

Plusieurs remarques :

- Le « Pour tout couple (P,E) » est implicite ;
- L'implication, est inversée (se lit de la gauche vers la droite) et se note : -
- Le "et" logique est noté " , " .

Question 4 : Complète la base de faits et règles avec le prédicat **mere/2**, puis pose les questions suivantes :

- Est-ce-que Gabriel est la mère de Raphaël ?
- Qui est la mère de Raphael ?

- De qui Emma est-elle la mère ?
- Qui est mère de qui ?

Question 5 : Complète la base de faits et règles avec les prédicats (ici des règles) suivants :

- pere(P,E) qui signifie que P est le père de E ;
- soeur(P,X) : P est la sœur de X ;
- grand_pere(P,X) : P est le grand-père de X.

Puis teste-les. Tu peux définir d'autres règles (frère, grand-mère, oncle ...)

Tu peux enregistrer ton programme dans ton dossier sous le nom C68-arbre_genealogique.pl : onglet File puis Download.

➔ Exercice 4 : Cryptarithme

1. Chaque lettre représente un chiffre et un seul. Aucun terme ne commence par 0.

Trouve-la ou les solutions à :

$$\begin{array}{r} \text{MOT} \\ + \text{MOT} \\ \hline + \text{MOT} \\ \hline = \text{BLA} \end{array}$$

2. Recopie, essaie de comprendre et teste dans l'interface de [swish](#) le programme suivant qui permet de résoudre le problème précédent (puis enregistre-le sous C68-ex04_Q2.pl) :

```
chiffre(X) :- between(0,9,X).
solution([M,0,T,B,L,A]):-
    chiffre(M),M=\=0,
    chiffre(0),0=\=M,
    chiffre(T),T=\=0,T=\=M,
    chiffre(B),B=\=0,B=\=T,B=\=0,B=\=M,
    chiffre(L),L=\=B,L=\=T,L=\=0,L=\=M,
    chiffre(A),A=\=L,A=\=B,A=\=T,A=\=0,A=\=M,
    3*(M*100+0*10+T)=B*100+L*10+A.
```

Aides : - between est un prédicat prédéfini

- « =\= » signifie différent ; « =:= » signifie =.

Combien de solutions sont proposées par Prolog ?

3. Même exercice avec
- | | |
|---------|----------|
| a) NEUF | b) CHIEN |
| + UN | + CHASSE |
| + UN | = GIBIER |
| = ONZE | |

Enregistre-les sous C68-ex04_Q3-a.pl et C68-ex04_Q3-b.pl.

Remarque : on aurait pu créer un prédicat tous_différents.

➡ Exercice 5 : Le code secret

Le code secret d'un coffre-fort est un carré parfait à **cinq chiffres**. Le chiffre des milliers (M), celui des dizaines (D), celui des unités (U) et celui des centaines (C), dans cet ordre, sont consécutifs : $D = M + 1$; $U = D + 1$; etc. La somme de tous les chiffres est 27.

Écrire le prédicat Prolog `code_secret/1` qui trouve le code secret correspondant à la description donnée ci-dessus puis enregistre-le sous `C68-ex05.pl`.

Aides : `sqrt(X)` est une fonction arithmétique qui calcule la racine carrée
`integer(X)` renvoie la valeur entière la plus proche du réel X

➡ Exercice 6 : Récursivité !

1. Bien sûr, Prolog permet la récursivité. C'est d'ailleurs la seule façon d'itérer. Voici par exemple une façon de calculer les termes de la suite de Fibonacci en Prolog :

```
fibonacci(1, 0).
fibonacci(1, 1).
fibonacci(U, N):- N>1, N1 is N-1, N2 is N-2, fibonacci(U1, N1), fibonacci(U2, N2), U is U1 + U2.
```

Recopie le code et teste-le dans l'interface de [swish](#) pour connaître U_6 .

Enregistre-le sous `C68-ex06-1.pl`.

2. Écrire le programme qui permet de calculer les termes de la suite (u_n) définie par $u_0 = 5$ et pour tout entier n $u_{n+1} = 3u_n - 7$.

Enregistre-le sous `C68-ex06-2.pl`.